

North Bengal International University, Rajshahi.



LAB REPORT

Course Title: Neural Network & Fuzzy System Sessional.

Course Code: CSE 0613-4222

Experiment No: 04

Name of Experiment: Effect of Learning Rate on Artificial Neural Network Performance.

SUBMITTED BY:

MD. RAZAB ALI
ID: 0712340012101011
DEPARTMENT OF CSE
7TH SEMESTER
BATCH-26TH
NBIU, RAJSHAHI.

SUBMITTED TO:

SAKURAH ISMAIL
LECTURER
DEPARTMENT OF CSE
NBIU, RAJSHAHI.

SUBMISSION DATE: 17 AUG 2026

Name of Experiment: Effect of Learning Rate on Artificial Neural Network Performance.

Theory: An Artificial Neural Network (ANN) is a machine learning model that learns patterns from data. In this lab, a simple ANN (784→128→64→10) is used to classify handwritten digits from the MNIST dataset. The **learning rate** controls how much the model updates its weights during training. Four learning rates (0.0001, 0.001, 0.01, and 0.1) are tested while keeping all other parameters the same. The training loss and test accuracy are plotted on the same graphs to compare the performance of each learning rate.

Finally, the trained model is used to show predictions and understand how the learning rate affects ANN performance.

Code:

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt
import numpy as np

# Set random seed for reproducibility
torch.manual_seed(42)
np.random.seed(42)

# Data Preparation (MNIST Dataset)
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])

train_dataset = datasets.MNIST(root='./data', train=True, download=True,
transform=transform)
test_dataset = datasets.MNIST(root='./data', train=False, download=True,
transform=transform)

train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=1000, shuffle=False)

# Define the ANN Architecture (784 -> 128 -> 64 -> 10)
class ANN(nn.Module):
    def __init__(self):
        super(ANN, self).__init__()
        self.flatten = nn.Flatten()
        self.fc1 = nn.Linear(784, 128)
        self.relu1 = nn.ReLU()
        self.fc2 = nn.Linear(128, 64)
        self.relu2 = nn.ReLU()
        self.fc3 = nn.Linear(64, 10)

    def forward(self, x):
        x = self.flatten(x)
        x = self.fc1(x)
        x = self.relu1(x)
        x = self.fc2(x)
        x = self.relu2(x)
        x = self.fc3(x)
        return x

# Training and Evaluation Function
def train_and_evaluate(lr, epochs=5):
    print(f"\n--- Training with Learning Rate: {lr} ---")
    model = ANN()
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=lr)
    train_losses = []
    test_accuracies = []
```

```

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)

for epoch in range(epochs):
    model.train()
    running_loss = 0.0
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)

        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

    running_loss += loss.item()

    epoch_loss = running_loss / len(train_loader)
    train_losses.append(epoch_loss)

    # Evaluate on test set
    model.eval()
    correct = 0
    total = 0
    with torch.no_grad():
        for images, labels in test_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            _, predicted = torch.max(outputs.data, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()

    accuracy = 100 * correct / total
    test accuracies.append(accuracy)
    print(f"Epoch [{epoch+1}/{epochs}], Loss: {epoch_loss:.4f}, Test
Accuracy: {accuracy:.2f}%")

    return model, train_losses, test accuracies

# Experiment with Learning Rates
learning_rates = [0.0001, 0.001, 0.01, 0.1]
results = {}

epochs = 5
for lr in learning_rates:
    model, losses, accuracies = train_and_evaluate(lr, epochs=epochs)
    results[lr] = {
        'model': model,
        'losses': losses,
        'accuracies': accuracies
    }

# Plotting Results
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
for lr in learning_rates:
    plt.plot(range(1, epochs+1), results[lr]['losses'], marker='o',
label=f'LR: {lr}')
plt.title('Training Loss across Learning Rates')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)

plt.subplot(1, 2, 2)
for lr in learning_rates:
    plt.plot(range(1, epochs+1), results[lr]['accuracies'], marker='s',
label=f'LR: {lr}')

```

```

plt.title('Test Accuracy across Learning Rates')
plt.xlabel('Epoch')
plt.ylabel('Accuracy (%)')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()

# Show Model Predictions (using LR = 0.001 model)
print("\n--- Model Predictions Demonstration (LR = 0.001) ---")
best_model = results[0.001]['model']
best_model.eval()
device = next(best_model.parameters()).device

dataiter = iter(test_loader)
images, labels = next(dataiter)
with torch.no_grad():
    outputs = best_model(images.to(device))
    _, preds = torch.max(outputs, 1)

fig, axes = plt.subplots(2, 5, figsize=(10, 5))
for i, ax in enumerate(axes.flatten()):
    ax.imshow(images[i].squeeze(), cmap='gray')
    pred = preds[i].item()
    true = labels[i].item()
    color = 'green' if pred == true else 'red'
    ax.set_title(f"Pred: {pred}\\nTrue: {true}", color=color)
    ax.axis('off')
plt.tight_layout()
plt.show()

```

Results:

--- Training with Learning Rate: 0.0001 --- Epoch [1/5], Loss: 0.6241, Test Accuracy: 91.60% Epoch [2/5], Loss: 0.2718, Test Accuracy: 93.32% Epoch [3/5], Loss: 0.2201, Test Accuracy: 94.07% Epoch [4/5], Loss: 0.1866, Test Accuracy: 95.01% Epoch [5/5], Loss: 0.1624, Test Accuracy: 95.45%	--- Training with Learning Rate: 0.01 --- Epoch [1/5], Loss: 0.2747, Test Accuracy: 93.66% Epoch [2/5], Loss: 0.1938, Test Accuracy: 95.62% Epoch [3/5], Loss: 0.1613, Test Accuracy: 94.87% Epoch [4/5], Loss: 0.1465, Test Accuracy: 95.88% Epoch [5/5], Loss: 0.1424, Test Accuracy: 96.20%
--- Training with Learning Rate: 0.001 --- Epoch [1/5], Loss: 0.2706, Test Accuracy: 95.77% Epoch [2/5], Loss: 0.1109, Test Accuracy: 96.82% Epoch [3/5], Loss: 0.0792, Test Accuracy: 96.94% Epoch [4/5], Loss: 0.0587, Test Accuracy: 97.33% Epoch [5/5], Loss: 0.0488, Test Accuracy: 97.56%	--- Training with Learning Rate: 0.1 --- Epoch [1/5], Loss: 2.7851, Test Accuracy: 19.15% Epoch [2/5], Loss: 2.3547, Test Accuracy: 11.35% Epoch [3/5], Loss: 2.3106, Test Accuracy: 9.74% Epoch [4/5], Loss: 2.3100, Test Accuracy: 10.28% Epoch [5/5], Loss: 2.3097, Test Accuracy: 11.35%

Table 4.1: Epoch-wise Training Loss and Test Accuracy for Different Learning Rates

Output:

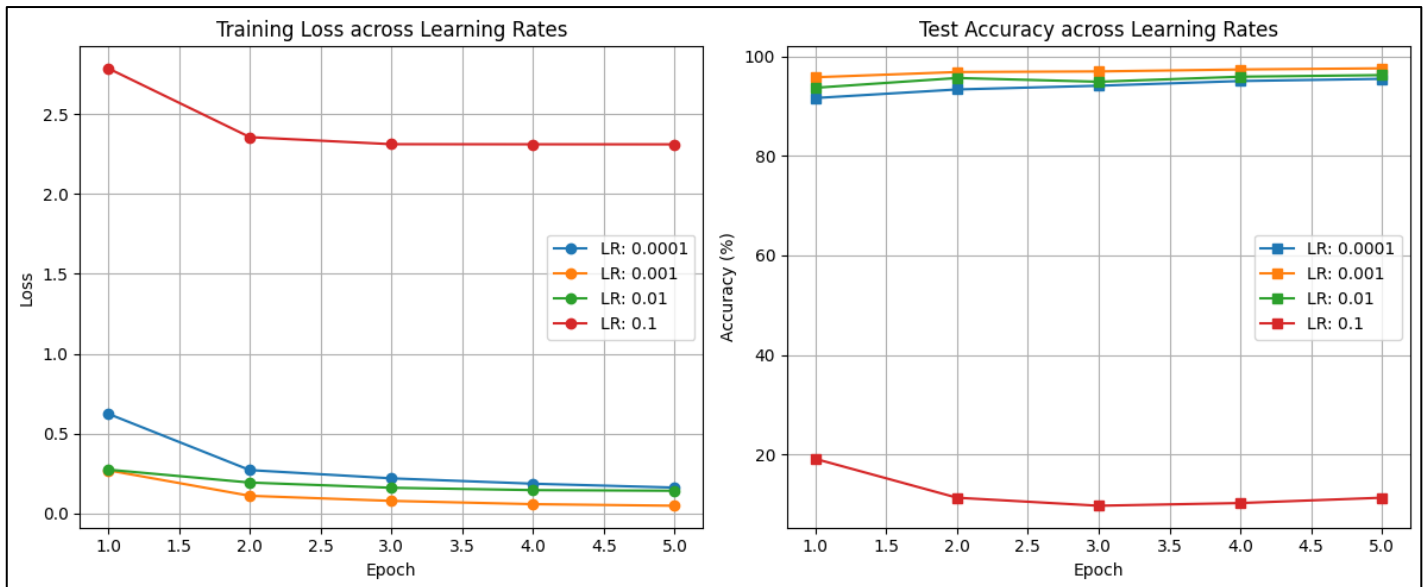


Fig. 4.1: Training Loss and Test Accuracy for Different Learning Rates.

Model Predictions Demonstration (LR = 0.001)

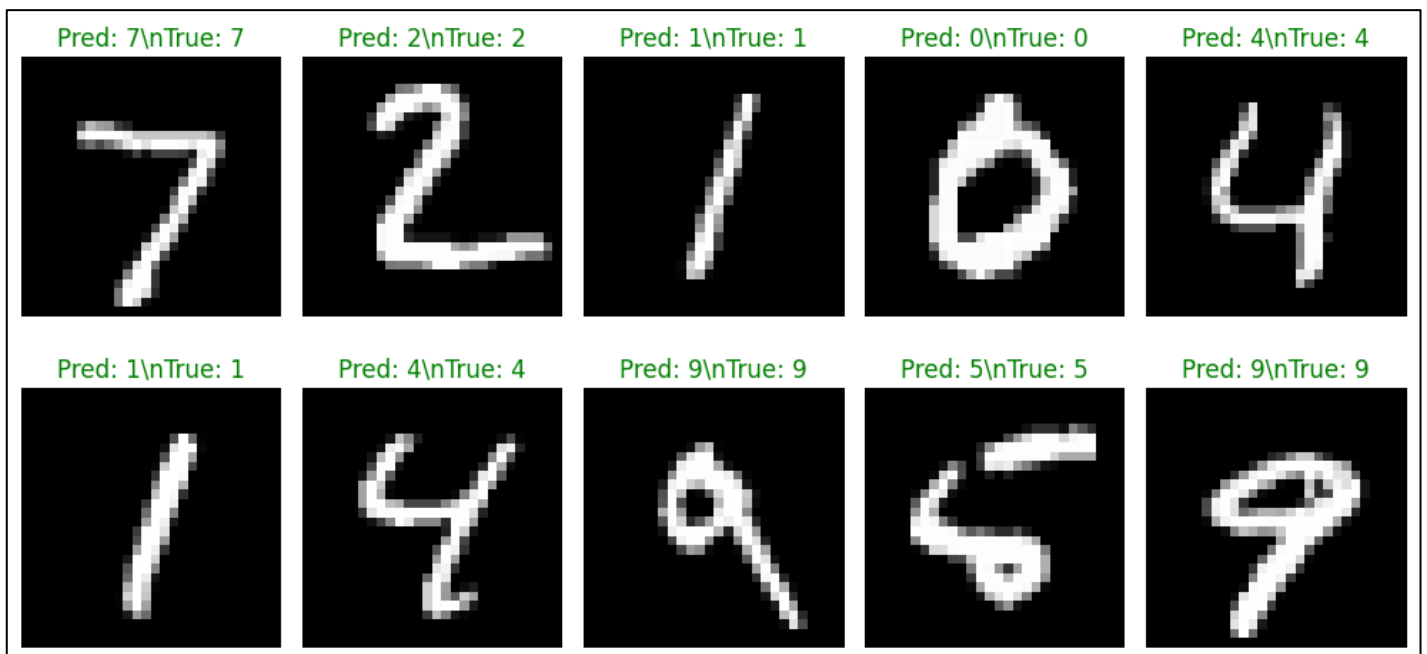


Fig. 4.2: ANN Model Prediction on MNIST Test Images

Comparison of ANN Performance for Different Learning Rates

Learning Rate	Training Loss Trend	Final Training Loss	Final Test Accuracy	Observation
0.0001	Gradually decreases	0.1624	95.45%	Stable but slow convergence. The model learns slowly because the update steps are small.
0.001	Decreases rapidly and smoothly	0.0488	97.56%	Best performance. It achieves the lowest loss and highest test accuracy.
0.01	Decreases with some fluctuation	0.1424	96.20%	Good performance, but the training process is less stable than 0.001.
0.1	Remains very high and nearly stagnant	2.3097	11.35%	The learning rate is too high, so the model fails to converge effectively.

Table 4.2: Comparison of ANN Performance for Different Learning Rates

Conclusion: This experiment examined how different learning rates affect the training and prediction performance of an Artificial Neural Network with the architecture $784 \rightarrow 128 \rightarrow 64 \rightarrow 10$ using the MNIST dataset. The same dataset, network structure, optimizer, loss function, batch size, and number of epochs were used for all experiments.

The results demonstrate that the learning rate has a significant effect on model convergence and accuracy. The learning rate 0.0001 produced stable but slow learning and achieved a final test accuracy of 95.45%. The learning rate 0.01 achieved 96.20% accuracy, although some fluctuation was observed during training. The learning rate 0.001 produced the best performance, with the lowest final training loss of 0.0488 and the highest test accuracy of 97.56%.

However, the learning rate 0.1 was too high for the model. It resulted in a high and nearly stagnant loss of 2.3097 and a low test accuracy of 11.35%, showing that the model failed to converge effectively. The prediction results from the model trained with learning rate 0.001 also show that the ANN can correctly classify handwritten digits from the MNIST test dataset.

Therefore, among the tested learning rates, **0.001 was the most suitable choice** for this ANN and MNIST classification task.

Colab Notebook Link: <https://shorturl.at/gh2El>